James Ryder
Prof. Lakisha Brooks
CIS-25: Programming Using C++
Fall 2026

Module09: Using AI to Expand Your Application

AI-Powered Feature: Error Explainer

- What it does:

When an C++ application fails to compile do to an error (a caught exception, a failed file load, invalid data, etc.), the Error Explainer sends the error and its surrounding context to an AI model. The model returns three things:

1. a plain-English explanation of what went wrong
2. the most likely cause
3. a suggested fix

The explanation is aimed at the developer during testing. A raw message like “std::out_of_range: vector::_M_range_check” gets translated to “The program tried to read item 5 of a list that only has 3 items, probably because the category menu index was not checked.”

- Data needed:

- **Error message and type:** the exception’s what() text and its class, such as std::out_of_range.
- **Location:** the function, file and line where the error was caught.
- **Recent log entries:** the last 20 or so lines the application logged before the error, to show what led up to it.
- **Program state:** a few relevant values, such as the user’s last input or the size of the container involved.
- **Source snippet (optional):** the few lines of code around the failure.

- What the user would see:

Upon encountering an error, a panel would appear below the usual error message. If the AI service is unavailable or its answer fails validation, the user sees only the original error message, so the feature never makes debugging worse.

Example:

```
ERROR: std::out_of_range in askUserForCategory() (main.cpp:171)

--- AI Explanation (82% confident) ---
What happened:      The program accessed position 5 in the category list,
                    which only has 5 items (valid positions are 0 to 4).
Likely cause:       The menu choice was used as an index without subtracting 1.
Suggested fix:      Use CATEGORIES[choice - 1] and check that choice is
                    between 1 and CATEGORIES.size().

AI suggestions may be wrong. Verify before changing code.
[1] Show raw log      [2] Continue      [3] Quit
```

- Where AI fits into the architecture:

The Error Explainer sits in the application's error-handling path:

User Request:	User selects an option labeled "Explain this error"
Prepare Data:	Build an error report from the exception and program state
AI Service/Model:	Send the report to Claude Code
Receive Result:	Get an explanation, the likely cause, suggested fix, and confidence percentage
Validate Result:	Check that the fields are not empty and confidence is in range
Display to User:	Show the explanation panel with a warning label, or fall back to the raw error

The rest of the application does not change. The only new parts are a small log buffer, the report builder, and the Claude Code interface.

- Things to consider:

At it's current stage, anything AI-powered is liable to make mistakes. Ironically, an AI-powered Error Explainer has the potential to create more errors than it solves. That's why it should only give suggestions, in addition to a percentage of confidence in its suggestion. That way, it's still up to the user to implement any actual changes.